PySpark: RDD vs DataFrame APIs

Quick study notes • API differences, shared operations and conversions

1. Core difference

RDD (Resilient Distributed Dataset): works with distributed Python objects such as tuples, lists and records. It provides lower-level control over record-level processing.

DataFrame: works with structured data arranged in named columns, similar to a relational table. It provides column-oriented operations and an optimized execution engine.

RDD	DataFrame
Record-oriented; lower-level transformations; useful for custom record processing.	Column-oriented; structured schema; preferred for most structured data engineering.

2. Commonly used RDD-specific functions

Function	Purpose	Example
map()	Transform each element	rdd.map(lambda x: x * 2)
flatMap()	Transform and flatten nested results	rdd.flatMap(lambda x: x.split(" "))
reduceByKey()	Aggregate values sharing a key	rdd.reduceByKey(lambda a,b: a+b)
groupByKey()	Group values for each key	rdd.groupByKey()
aggregateByKey()	Aggregate per key using a zero value and functions	rdd.aggregateByKey(0, seq, comb)
combineByKey()	Define custom per-key aggregation	rdd.combineByKey(...)
mapPartitions()	Process all records in a partition	rdd.mapPartitions(func)
mapPartitionsWithIndex()	Process partitions with their index	rdd.mapPartitionsWithIndex(func)
zipWithIndex()	Attach an index to each element	rdd.zipWithIndex()
cartesian()	Cartesian product of two RDDs	rdd1.cartesian(rdd2)
glom()	Collect each partition's elements into a list	rdd.glom()

Pair RDD note: reduceByKey(), groupByKey(), aggregateByKey() and combineByKey() are key-based operations used with RDDs of (key, value) pairs.

3. Commonly used DataFrame-specific functions

Function	Purpose	Example
select()	Choose columns	df.select("name", "salary")
withColumn()	Add or replace a column	df.withColumn("bonus", col("salary") * 1.1)
drop()	Remove columns	df.drop("salary")
withColumnRenamed()	Rename a column	df.withColumnRenamed("name", "emp_name")
selectExpr()	Use SQL expressions in column selection	df.selectExpr("salary * 1.1 AS bonus")
groupBy() / agg()	Group rows and aggregate values	df.groupBy("dept").agg(sum("salary"))
orderBy()	Sort rows	df.orderBy("salary")
dropDuplicates()	Remove duplicate rows or keys	df.dropDuplicates(["emp_id"])
fillna() / dropna()	Fill or remove null values	df.fillna(0) / df.dropna()
explode()	Expand array elements into rows	df.select(explode("skills"))
pivot()	Turn row values into columns	df.groupBy("dept").pivot("year")

Function	Purpose	Example
printSchema()	Display column names and data types	df.printSchema()
explain()	Display logical and physical plans	df.explain(True)

4. Functions available in both APIs

Some names exist in both APIs, but their input and output types differ. The API context determines which implementation is called.

Operation	RDD	DataFrame
filter()	rdd.filter(lambda x: x[2] == "IT")	df.filter(col("dept") == "IT")
distinct()	Distinct elements	Distinct rows
count()	Count elements	Count rows
collect()	Return Python elements to driver	Return Row objects to driver
take(n) / first()	Return first elements	Return first rows
union()	Combine RDDs	Combine compatible DataFrames
join()	Join pair RDDs by key	Join DataFrames using columns

Important: map() is a direct RDD transformation, not a DataFrame method. For DataFrames, use select(), withColumn(), SQL expressions or convert to an RDD with df.rdd when record-level RDD processing is genuinely needed.

5. Converting between RDD and DataFrame

You can convert between the two APIs. An RDD containing structured records can become a DataFrame; a DataFrame can expose its underlying RDD.

RDD → DataFrame

```
rdd = sc.parallelize([
(101, "Abhi", "IT"),
(102, "Rahul", "HR")
])
df = rdd.toDF(["emp_id", "name", "dept"])
df.show()
```

DataFrame → RDD

```
rdd = df.rdd
names = rdd.map(lambda row: row.name)
names.collect()
```

When to convert: Convert an RDD to a DataFrame when you need schema-aware operations and DataFrame/SQL optimizations. Convert a DataFrame to an RDD only when an RDD-specific operation is genuinely needed.

6. Quick comparison and practical guidance

RDD	DataFrame
Record-oriented API	Column-oriented API
map(), flatMap(), reduceByKey(), groupByKey(), mapPartitions()	select(), withColumn(), groupBy(), agg(), explode(), dropDuplicates()
Lower-level control over distributed objects	Structured schema and query optimization
Useful for custom record-level processing	Preferred for most structured data engineering workloads

Interview takeaway: RDDs provide lower-level record processing. DataFrames provide structured, column-based processing and are generally preferred for structured ETL. Some operations share names, but their semantics and return types depend on the API.

Memory cue: RDD = records and functions. DataFrame = columns and expressions.